


Lec 24
3 octo 2023

17 Nov, 2024

Hoisting Continued:-

↳ pchle Shreyance sir ke notes
pado (Lec 14 6 July 2024)

maine sirf important chizo ke
notes banaye.

Hoisting → accessing a variable/function before
its declaration.

①

```
b(); → l1  
console.log(a); → l2  
var a = 30;  
function b(){  
  console.log(c); → l3  
  var c = 100;  
}
```

⇒ o/p : undefined
undefined

l1 par hoisting ho rahi hai

l2 " " " " "

l3 " " " " "

②

```
let a = 10;  
console.log(a); → 11  
  
function b(){  
    console.log("aa dekhe zara kisme kitna hai dum")  
}  
  
b(); → 12
```

11 → yaha hoisting nahi ho saki.

12 → " " " " " "

a and function b both are declared before we access them.

O/p → 10
aa dekhe zara ...

③

```
let a; → 10  
console.log(a); → 11  
a=100;  
function b(){  
    console.log("aa dekhe zara kisme kitna hai dum")  
}  
  
b(); → 12
```

declaration

reinitialization

Kyunki
JS engine
ne khud se

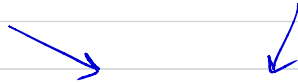
a ko undefined
kar diya kyunki
hamne koi value
assign nahi ki thi.

O/p → undefined
aa dekhe zara ...

11 → yaha hoisting nahi hui kyunki 10 par hamne let a declare kar diya. Since we did not assign anything, it will be assigned "undefined"

12 → hoisting nahi hui. Kyunki we tried to access the function after its declaration.

let a or let a = undefined

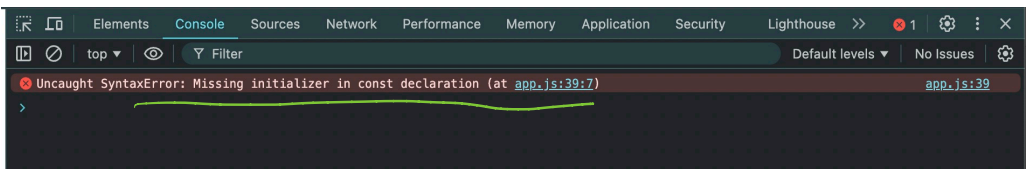


Both are same

Q guess the o/p :-

```
const a;  
console.log(a);  
function b(){  
  console.log("aa dekhe zara kisme kitna hai dum")  
}  
b();
```

error aayega → a ko saath saath initialise karna padega.



Q console.log(a);
let a = 1000;

o/p → error cant access a before initialization.

Pls check screenshots on the next page.

Key Concepts: Hoisting and `let` Declaration

1. Hoisting:

- In JavaScript, variables declared with `var`, `let`, and `const` are hoisted, meaning they are moved to the top of their scope during the **creation phase**.
- However, there's a key difference:
 - `var`: Variables are initialized with `undefined` during hoisting.
 - `let` and `const`: Variables are **hoisted** but are not initialized. Instead, they remain in a **temporal dead zone (TDZ)** from the start of the block until the line of code where they are declared and initialized.

2. Temporal Dead Zone (TDZ):

- For variables declared with `let` or `const`, accessing the variable **before its declaration** results in a `ReferenceError` because it is in the TDZ.

Execution Step-by-Step

Phase 1: Creation (Hoisting)

- The JavaScript engine parses the code and hoists the variable `a` to the top of its scope.
- Because `a` is declared with `let`:
 - `a` is hoisted but remains **uninitialized**.
 - It is in the **TDZ** and cannot be accessed until the `let a = 100;` line is reached.

Memory state during creation:

```
javascript
```

Copy code

```
a → Uninitialized (in TDZ)
```

Phase 2: Execution

1. `console.log(a);`
 - At this point, `a` is still in the TDZ because the declaration `let a = 100;` has not been executed yet.
 - Attempting to access `a` results in a `ReferenceError: Cannot access 'a' before initialization`.
2. `let a = 100;`
 - After this line, `a` is initialized to `100`, and it can be accessed normally.

Output

The code results in a `ReferenceError`:

```
javascript
```

Copy code

```
ReferenceError: Cannot access 'a' before initialization
```

Q guess the o/p :-

```
console.log(sam);  
a();  
  
var sam = 100;  
function a(){  
    console.log("inside a")  
}
```

O/p $\Rightarrow$ undefined
inside a

Q guess the O/p :-

```
console.log(sam);  
a();  
  
let sam = 100;  $\rightarrow$  l1  
function a(){  
    console.log("inside a")  
}
```

O/p $\Rightarrow$ error cant access ^{sam} before initialization

JS engine ko pata hai ki sam variable ko declare kiya gaya hai code me but kaha declare kiya gaya hai ye nahi pata. Jab take code ka execution ~~wo~~ l1 tak nahi pahuncha tab tak JS engine ko sam ki value nahi pata chalegi.

Note:- JS me error aapke ke baad code execution

ruk jayega .

```
console.log("mera"); → l1  
console.log(a); → l2  
let a; → l3  
console.log("tera"); → l4
```

l1 print ho jayega, l2 per error aayega & execution ruk jayega.

l3, l4 nahi chalenge.

Q Kya hoisting var, let, const teeno mei hoti hai?

Ans yes, hoisting teeno cases mei hoti hai.

But var ka behavior is

different than that of let & constant.

Q

```
console.log(a);  
let a
```

o/p → cannot access a
before initialization

Q

```
console.log(a);
```

o/p ⇒ Reference error:
a is not defined

↓ ↓
you can see difference b/w both statements of errors.

ek error keh raha hai a not defined,
dusra error keh raha hai cannot access
a before initialization.

a not defined → ye keh raha hai ki
a declare nahi kiya
gya.
hai

cannot access a
before initialization → ye keh raha hai ki a declared
hai but main access nahi kar
sakta. This means ki
let a was hoisted.

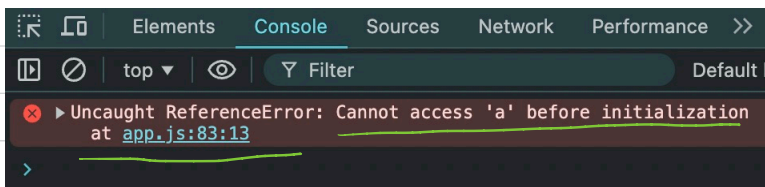
Teeno declarative mei let, const, var mei
hoisting aag hoti hai but op aag aag nahi hai.

let and const ki hoisting ko TDZ kehte hai.

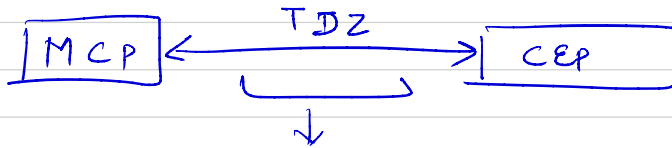
TDZ → Temporal Dead Zone

Q guess the op :-

```
console.log(a);  
const a = 100;
```



Memory Creation Phase and Code execution phase
ke beech mei ek jagah hoti jise TDZ khte
hai.



iss jagah per you cannot access let &
const variables

2d mt ye line $\Rightarrow$ In let and const, hoisting takes
place but acts differently i.e. in
TDZ.

{ 1.17.41 Break over }

In JS, var declaration is hoisted with a default initialization of undefined.

let and const are hoisted too but without any initialization.

var Declarations:

Hoisting: Yes, var declarations are hoisted.

Initialization: During hoisting, var variables are initialized with undefined.

let and const Declarations:

Hoisting: Yes, let and const declarations are also hoisted.

Initialization: let and const variables are not initialized during hoisting. They remain uninitialized and are in the Temporal Dead Zone (TDZ) until the execution reaches the line where they are declared.

Key Points About TDZ

1. Scope Bound:

- The TDZ starts from the top of the scope where the variable is declared.

2. Access During TDZ:

- Accessing the variable in the TDZ throws a `ReferenceError`.

3. Initialization Ends TDZ:

- The TDZ ends as soon as the variable is initialized.

Visualizing TDZ

javascript

Copy code

```
{  
  // TDZ starts for 'x'  
  console.log(x); // ❌ ReferenceError: Cannot access 'x' before initialization  
  let x = 10;     // TDZ ends here  
  console.log(x); // ✅ 10  
}
```

- Start of TDZ:** The moment the scope (block in this case) begins.
- End of TDZ:** When the variable `x` is initialized at `let x = 10;`.

Jaise hi code ka scope shuru hoga, waise hi TDZ shuru hoga. And jab execution declaration wali line wali par aayega. TDZ khatam ho jayega.

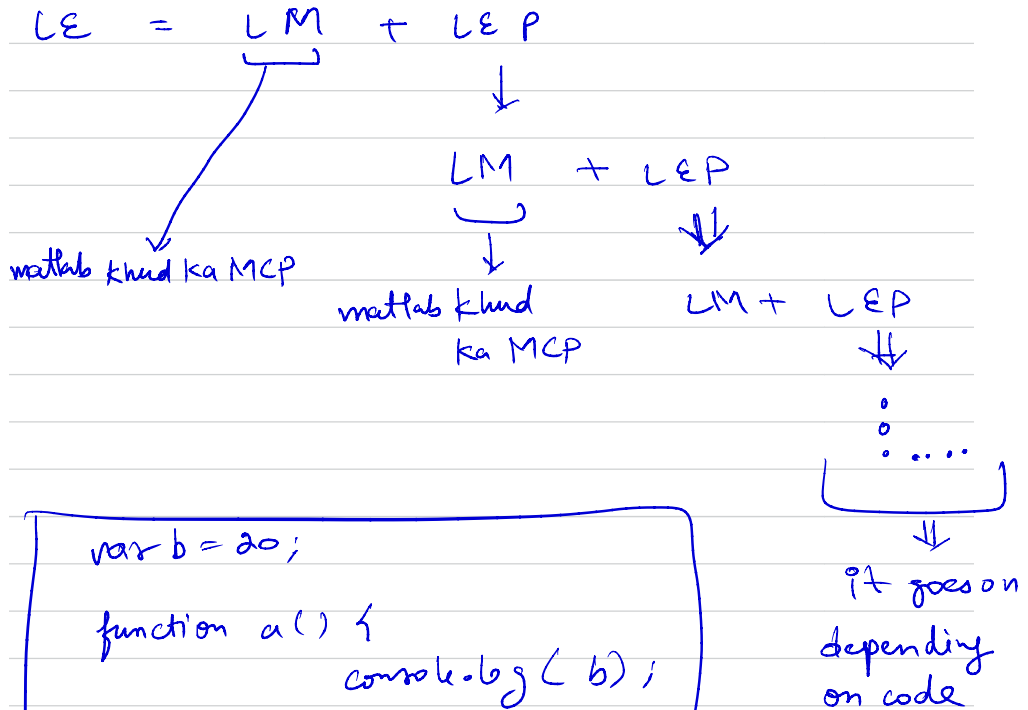
Note:- TDZ sirf hoisting ke case mei hoga.

```
let u = 10;  
console.log(u);
```

→ iss example mei hoisting nahi hui, isliye TDZ nahi baneگا.

Scope :- scope matlab lexical environment in JS.

Scope / lexical Environment $\Rightarrow$ Local memory + lexical environment of parent



```
var b = 20;  
  
function a() {  
    console.log(b);  
}  
  
a();
```

↳ Draw Global Execution Content of this code.

Note: Scope hi lexical environment hai.

MCP	CEP
b = undefined function a definition	



MCP	CEP
b = undefined 20 function a definition	b = 20 function call hua



MCP	CEP				
b = undefined 20 function a definition	<table> <tr> <th>MCP</th><th>CEP</th></tr> <tr> <td></td><td>console b = 20</td></tr> </table>	MCP	CEP		console b = 20
MCP	CEP				
	console b = 20				

Notice karo ki function a ke Execution content ke memory mei b variable nahi diya hua hai. Isliye parent se b ki value li jayegi. yahi hai lexical environment = LMemory + LE of parent


```
var b = 20;
```

```
function a(){  
  console.log(b + "first");
```

```
  function d(){  
    console.log(b);  
  }
```

```
  d();
```

```
}
```

```
a();
```

→ l2

o/p ⇒ 20first
20

yaha agar hum notice kare l2 par bhi we are able to access b. Iska matlab iss case mei you can case b globally.

LE = LM + LEP

└──┘

↓

it means lexical environment of parent / ultimate parent.

Q guess the o/p :-

```
function outer(){  
  var b = 10;
```

```
  inner();
```

```
  function inner(){  
    console.log(b);
```

```
  }
```

```
}
```

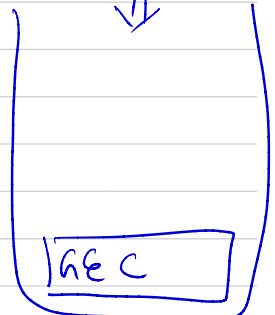
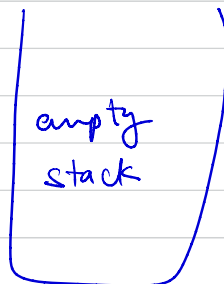
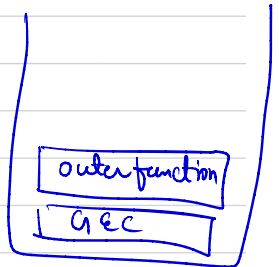
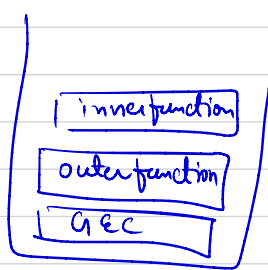
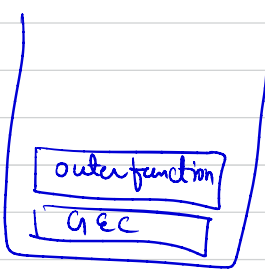
```
outer();
```

o/p ⇒

10

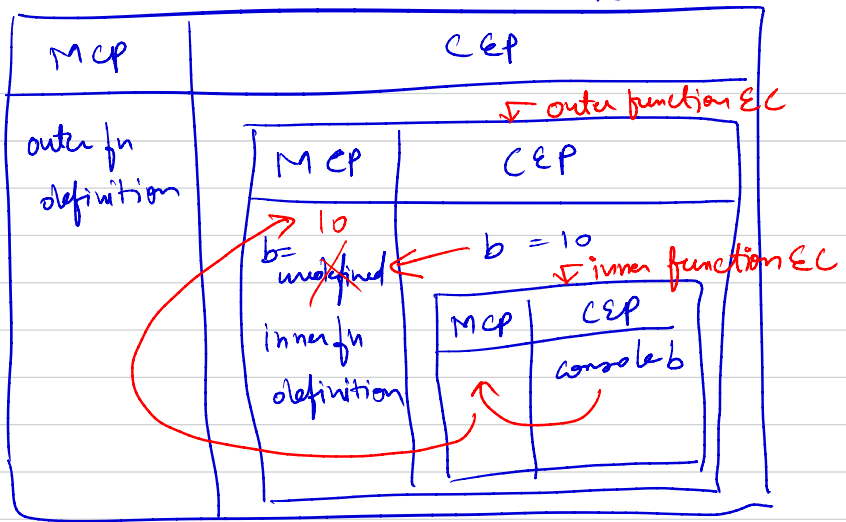
call stack:-

Note:- Call stack mei
sirf execution
content aata hai.



you can also make global execution content.

GEC



console :- 10

- ① pehle MCP mei outer fn ka defn aaya
- ② Then outer fn call hua
- ③ Ab naya EContext banega
- ④ MCP mei b ~ undefined, inner function aa gyi
- ⑤ CEP shuru hote hi b = 10 ho gaya, then inner function call ho gaya.
- ⑥ Ab naya Ene context banega, MCP mei kuch nahi aayega bcoz there is no variable declared inside inner function. b=10 console ho jayega. b ki value uske khud ke local memory (MCP se nahi aaygi balki parent ke MCP se aaygi (shown through arrow)). Inner function ka EC khatam ho gaya, then outer function ka EC khatam ho gaya, then GEC khatam ho gaya, call stack khaali ho jayega ab.

Q

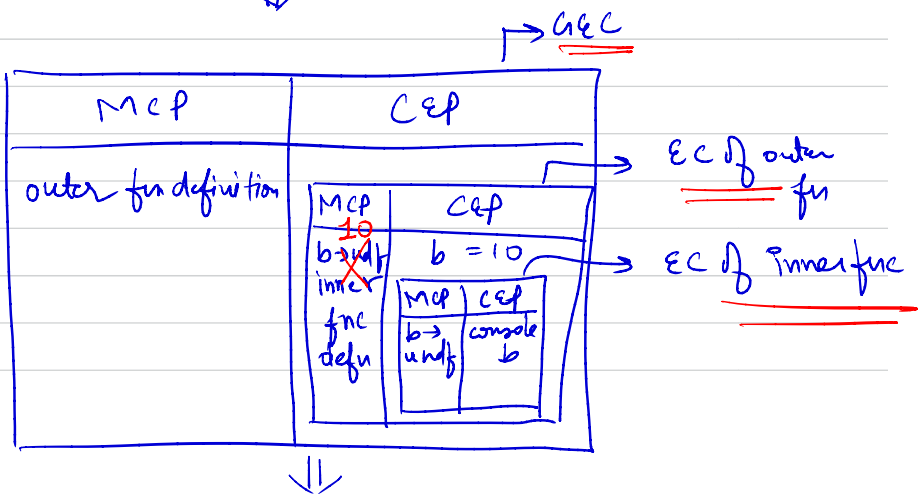
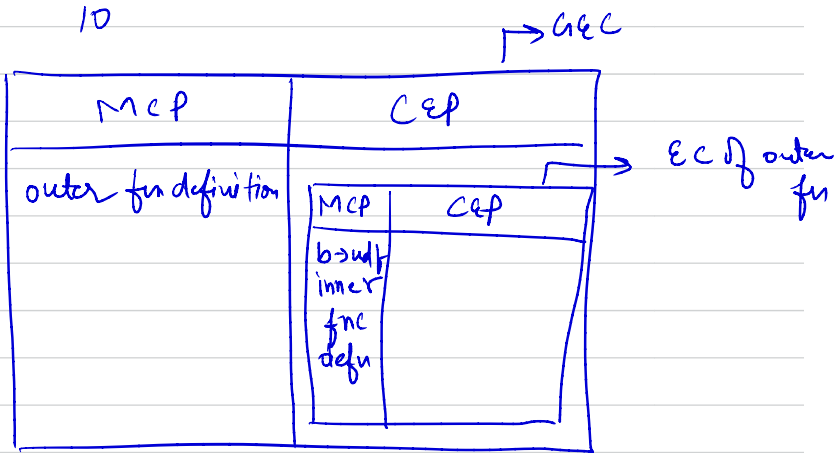
```
function outer(){
  var b = 10;

  inner();
  function inner(){
    var b = 1000;
    console.log(b);
  }

  console.log(b);
}

outer();
```

O/p ⇒ 1000
10



MCP	CEP
b → undef 1000	b → console 1000

EC of inner func
EC of inner func

inner function ke EC mei b is undefined initially, then it becomes 1000, then b is console.

0

```
function outer(){
  var b = 10;

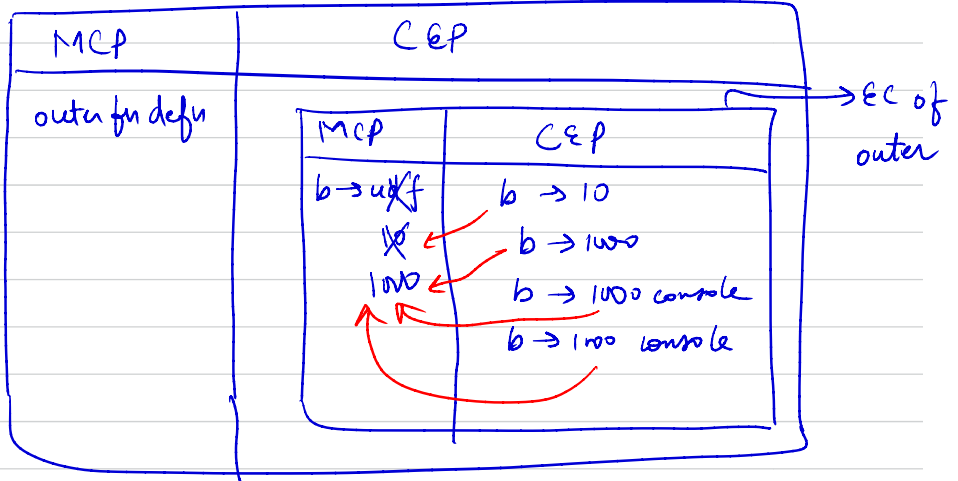
  {
    var b = 1000;
    console.log(b);
  }

  console.log(b);
}

outer();
```

O/p $\Rightarrow$ 1000
1000

$\rightarrow$ GEC



Note:- outer function ke MCP phase mei you dont go inside $\{ \}$ block. Ham sirf usi ko evaluate consider karenge MCP mei jo hamara ~~code~~ main code hai naaki koi block ya function.

$\{ \}$ $\rightarrow$ ye block C&P phase mei evaluate hoga. C&P phase mei jab hum block mei ghusenge,

tab new EContext nahi banega; New EContext

- ext sirf function ke call hone par banata hai. yaha b update ho jayega from 10 to 1000.

$\rightarrow$ ye explanation galat hai a/c to my browser & gpt. check page = 20.

Scope :- let and const dono hi hoist hote hai in TDZ.

	let & const	var
Hoisting	① Yes in TDZ	① Normal hoisting hoti hai
Scope	scope → block level scope → script / local level scope	scope → functional / local → global

Read notes → Lec 14 6 July 2024
of scope Shreyansh sir Page 25

JS ka code tab tak chalega jab tak call stack empty na ho jaye.

2:18:00 time se sir ne browser me call stack & scope dikhaya by attaching breakpoint in the code. Must watch the video when preparing for interview. Mainu notes isliye nahi banaye kyunki wo chij video dekhke hi samajh aayegi.

Imp

```
Code:- for( var i = 0; i < 3; i++)
{
    console.log(i);
}
console.log(i);
```

O/p: 0
1
2
3
(~~no~~ no error)

^{mei} iss code error isliye nahi aaya kyunki var i ka scope ^ global hai. var i tabhi khatam hoga jab GC khatam hoga.

Q

```
var a = 10;
{
    var b = 20;
    console.log(b);
}
console.log(b);
console.log(a);
```

→ Mera browser ke mutabik iss code ka GC jab khatam hoga tab a & b dono undefined honge but Samarth sir ne bola tha ki b block mei hai isliye vo MCP mei nahi ~~log~~ aayega.

↓
chatgpt bhi yahi kehتا.

